

Arithmetic Operations

Computer Engineering 1

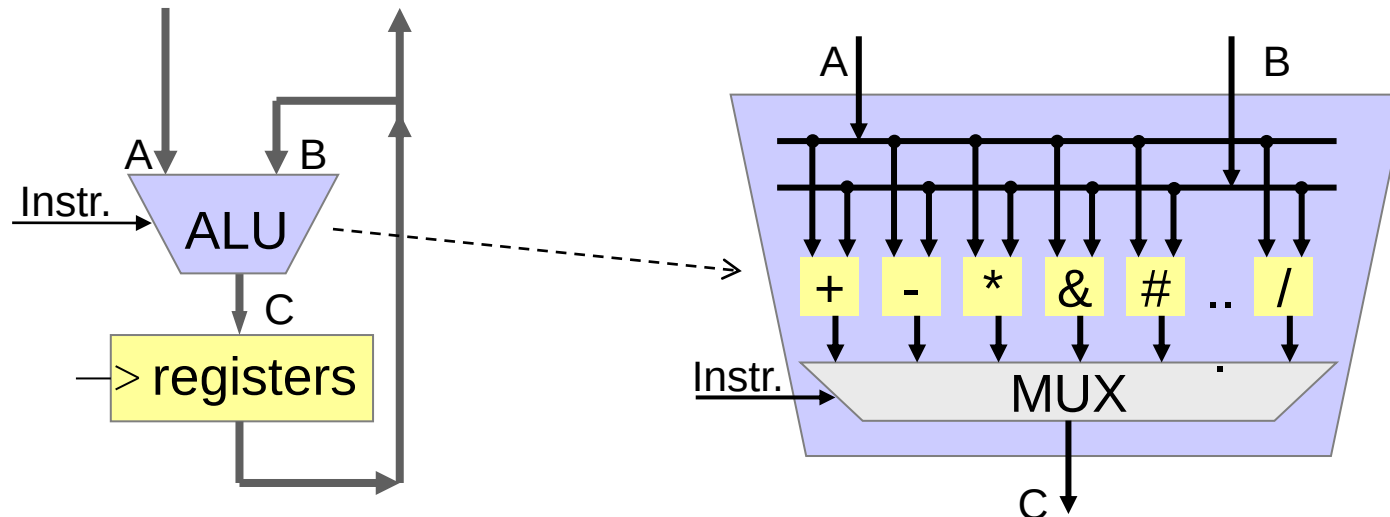
Motivation



"I call them numbers, you can add them, subtract them, multiply them, divide them ... find their square root ..."

Motivation

- Instructions to process data in the ALU
 - **arithmetic** Addition, Subtraction, Multiplication, Division
 - **logic** NOT, AND, OR, XOR
 - **shift** Shift left/right. Fill with 0 or MSB
 - **rotate** Cyclic shift left/right: What drops out enters on the other side



Motivation

- How often is the for loop executed?
 - For `#define INCREMENT 1`
 - For `#define INCREMENT 10`

```
...
int main(void) {
    uint8_t uc;
    uint32_t count = 0;

    for (uc = 0; uc < 255; uc += INCREMENT) {
        printf("hello again %d \n",uc);
        count++;
    }
    printf("Loop executed %d times\n",count);
}
```

Agenda

- Cortex-M0
 - Data flow
 - Flags
 - Overview of arithmetic instructions
- Add Instructions Cortex-M0
- Negative Numbers
- Addition
- Subtraction
- Subtract Instructions Cortex-M0
- Multi-Word Arithmetic
- Multiplication

Learning Objectives

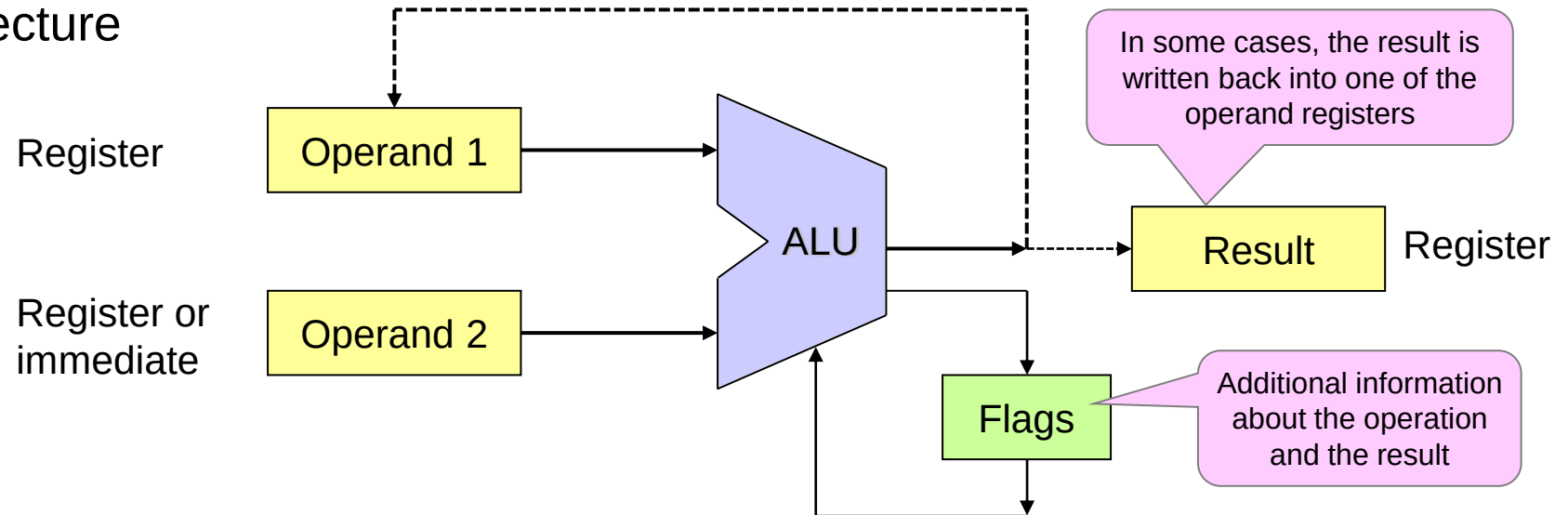
At the end of this lesson you will be able

- to enumerate and apply the Cortex-M0 arithmetic instructions
- to interpret Cortex-M0 assembly programs with arithmetic instructions
- to enumerate and explain the meaning of the Cortex-M0 Flags (N, Z, C, V)
- to carry out additions and subtractions of signed and unsigned integers and to explain the operations with the circle of numbers
- to calculate and interpret carry/borrow and overflow/underflow
- to determine (with the help of documents) the state of Cortex-M0 Flags (N, Z, C, V) after an arithmetic instruction
- to describe how addition and subtraction are done in hardware (in the ALU)
- to program integer calculations with operands that exceed the number of bits available in the ALU
- to explain how numbers in two's complements representation are multiplied

Data flow

Example: Cortex-M0

- Operands and results stored in registers
 - Exception: Immediate operations with data in OP-code
 - Load/store architecture



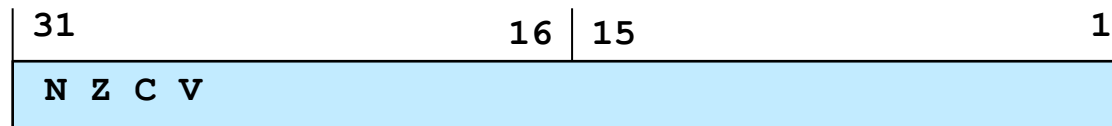
Examples

<code>adds</code>	<code>r0, r0, r1</code>	@ <code>r0 = r0 + r1</code>	result back in r0
<code>adds</code>	<code>r0, #0x34</code>	@ <code>r0 = r0 + 0x34</code>	
<code>subs</code>	<code>r3, r4, #5</code>	@ <code>r3 = r4 - 0x05</code>	result in other reg

Flags

Application Program Status Register

- APSR: Application Program Status Register



Flag	Meaning	Action	Operands
Negative	MSB = 1	N = 1	signed
Zero	Result = 0	Z = 1	signed , unsigned
Carry	Carry	C = 1	unsigned
Overflow	Overflow	V = 1	signed

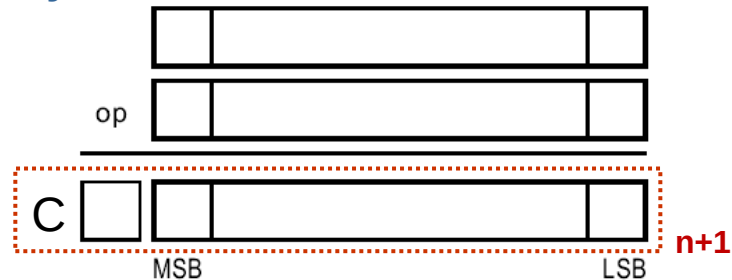
- Processor does not know whether the user is working with unsigned (C) or with signed (V) numbers
→ Processor therefore always calculates C and V!

Flags

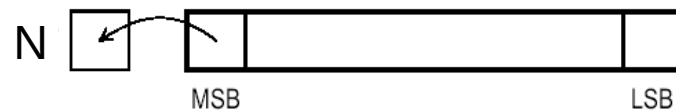
Modification of flags

- Instructions ending with ,s' allow modification of flags
- Examples: **movs**, **adds**, **subs**, ...

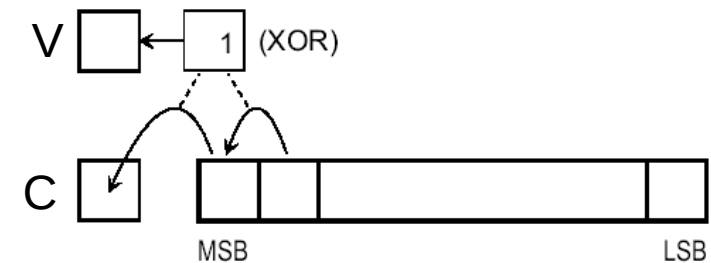
Carry



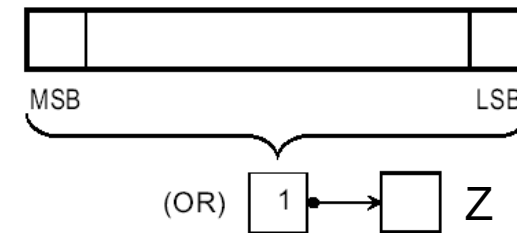
Negative



Overflow



Zero



Flags

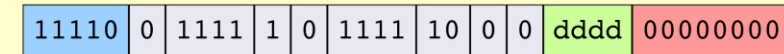
Instructions to access APSR

- **mrs** Move to register from special register (APSR)
- **msr** Move to special register (APSR) from register
- 32-bit opcode

mrs rd, apsr

T1

rd ∈ { r0..r11 }



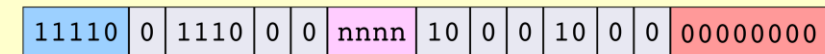
rd = apsr;

*'mrs' can move other special registers, but we use only **apsr**. This would change the red coloured bits.*

msr apsr, rn

T1

rn ∈ { r0..r11 }



apsr = rn;

*'msr' can move other special registers, but we use only **apsr**. This would change the red coloured bits.*

Arithmetic Instructions

Overview Cortex-M0

Mnemonic	Instruction	Function
add / adds	Addition	$A + B$
adcs	Addition with carry	$A + B + c$
adr	Address to Register	$PC + A$
sub / subs	Subtraction	$A - B$
sbc	Subtraction with carry (borrow)	¹⁾ $A - B - \text{NOT}(c)$
rsbs	Reverse Subtract (negative)	$-1 \cdot A$
mul	Multiplication	$A \cdot B$

¹⁾ borrow = NOT(c)

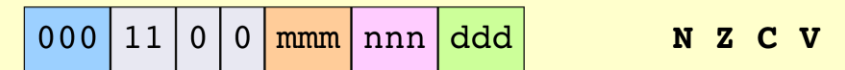
Add Instructions Cortex-M0

adds (register)

- Update of flags
- Result and two operands
- Only low register

adds rd, <rn>, rm *T1*

rd, rn, rm ∈ { r0..r7 }



rd = rn + rm; or rd += rm;

00000002	18D1	adds	r1,r2,r3	
00000004	1889	adds	r1,r1,r2	
00000006	1889	adds	r1,r2	@ the same (dest = r1)
00000008		@adds	r9,r2	@ not possible (high reg)
00000008		@adds	r1,r10	@ not possible (high reg)

Add Instructions Cortex-M0

add (register)

- No update of Flags
- High or low register
- <Rdn> → result and operand
 - I.e. same register for operand and result!

add rd, rm T2
rd, rm { r0..r11 }

010001	00	d	mmmm	ddd
--------	----	---	------	-----

rd = rd + rm;

00000008	4411	add	r1,r1,r2	@ low regs
0000000A	44D1	add	r9,r9,r10	@ high regs
0000000C	44D1	add	r9,r10	@ the same (dest = r9)
0000000E	4411	add	r1,r1,r2	
0000000E		@add	r1,r2,r3	@ not possible

Add Instructions Cortex-M0

adds (immediate) – T1

- Update of flags
- Two different¹ low registers and immediate value 0 - 7

```
adds rd, rn, #imm3                                T1
rd, rn ∈ { r0..r7 }
imm3 ∈ { 0..7 }
```

000	11	1	0	iii	nnn	ddd	
-----	----	---	---	-----	-----	-----	--

N Z C V

rd = rn + imm3;

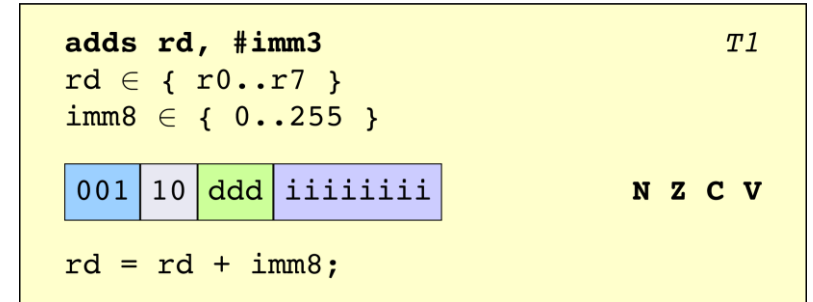
00000010	1D63	adds	r3,r4,#5	
00000012		@adds	r3,r4,#8	@ out of range immediate
00000012		@adds	r10,r11,#5	@ not possible (high reg)

¹ If the same register is used for Rd and Rn, the assembler will choose the encoding T2 on the next slide

Add Instructions Cortex-M0

adds (immediate) – T2

- Update of flags
- Low register with immediate value 0 - 255d
- <Rdn> → Result and operand in same register



00000012	33F0	adds	r3,r3,#240	
00000014	33F0	adds	r3,#240	@ the same (dest = r3)
00000016		@adds	r8,r8,#240	@ not possible (high reg)
00000016		@adds	r3,#260	@ out of range immediate

Add Instructions Cortex-M0

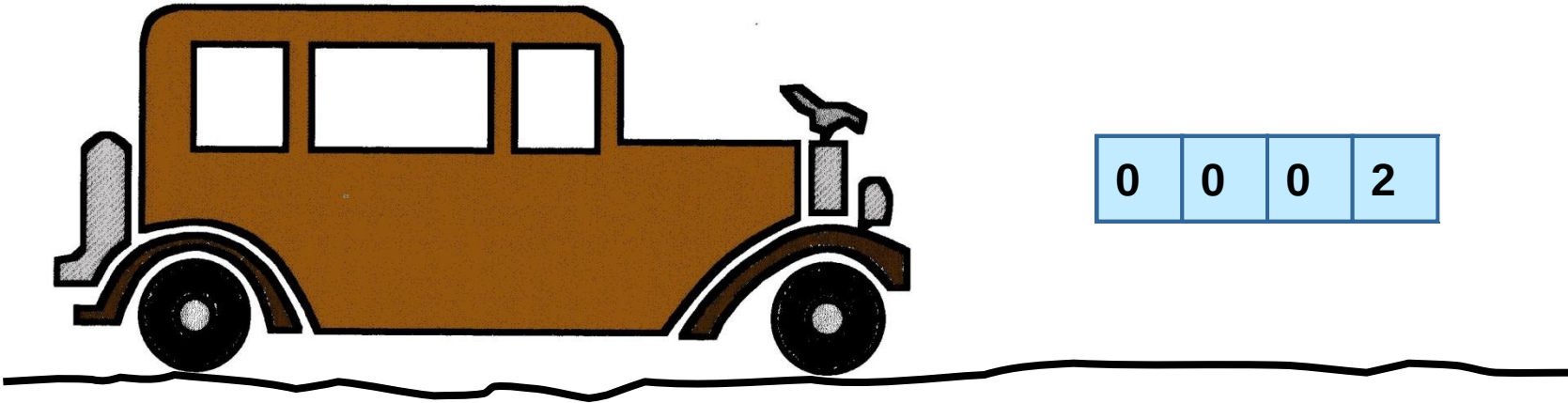
add / adds (Summary)

- add Flags not changed
- adds Flags changed according to Operation Result

Instr	Rd	Rn	Rm	imm	Restrictions
add	r0-r15	r0-r15	r0-r15	-	Rd and Rn must specify the same register. Rn and Rm must not both specify the PC (r15)
adds	r0-r7	r0-r7	-	0 - 7	-
adds	r0-r7	r0-r7	-	0 - 255	Rd and Rn must specify the same register
adds	r0-r7	r0-r7	r0-r7	-	-

Negative Numbers

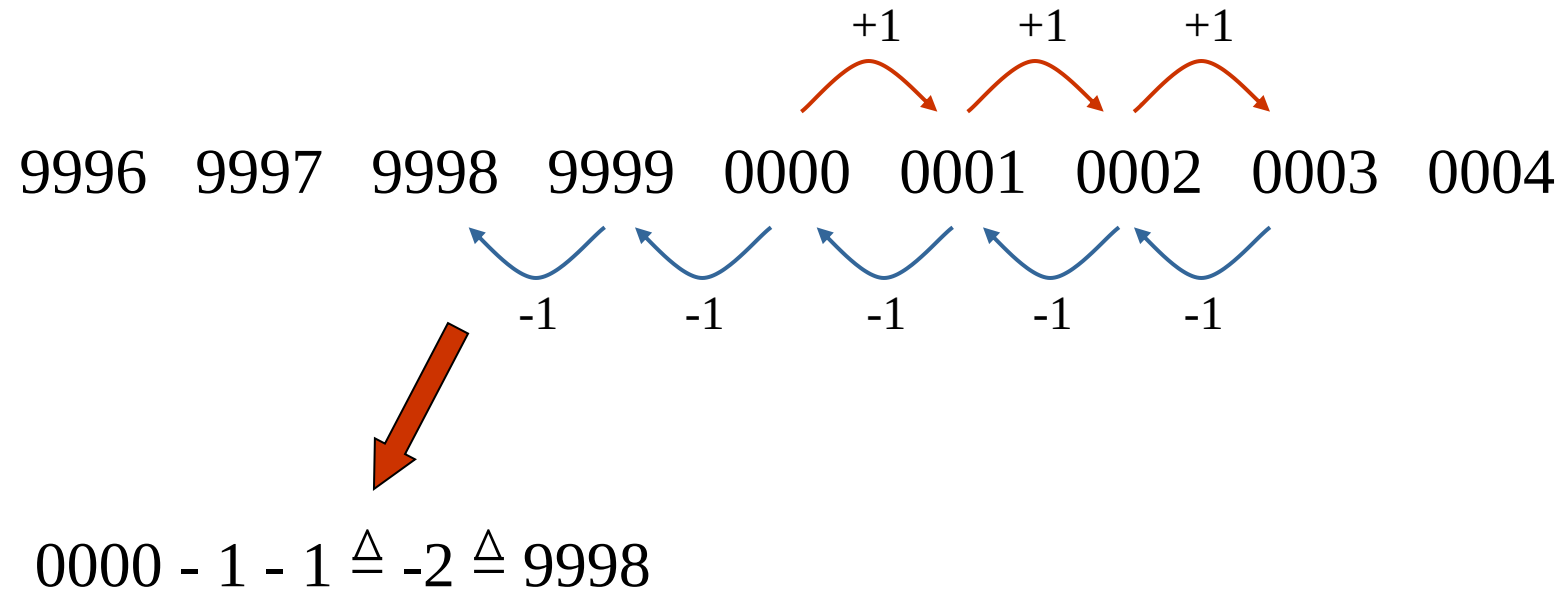
Lord Ardry's new Rolls Royce (1)



Negative Numbers

Lord Ardry's new Rolls Royce (2)

- What happened?



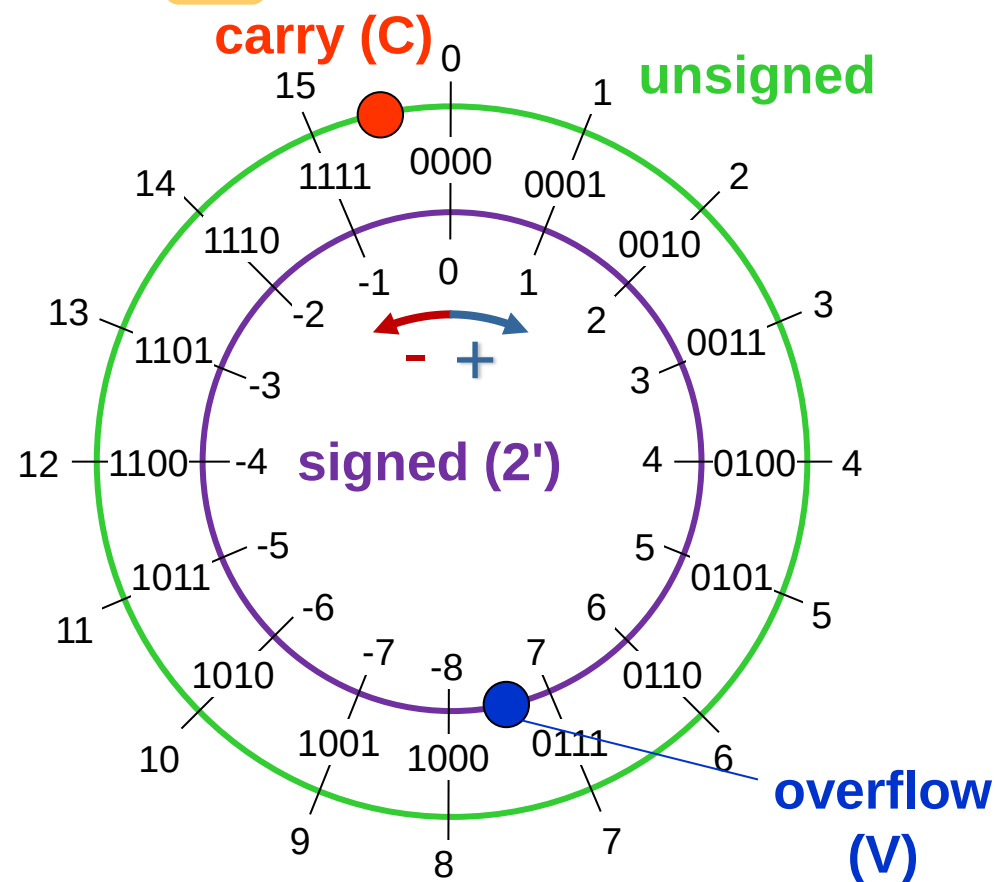
possible because

- number of digits is finite
- respectively given by the word size

Negative Numbers

signed $-b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0$

unsigned $+b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0$

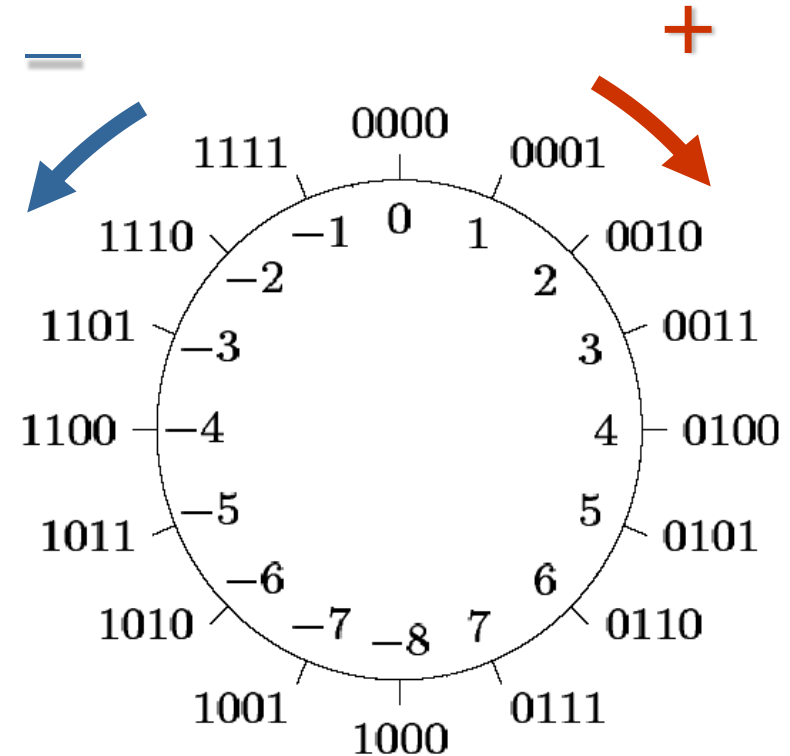


binary	unsigned	signed 2'-Compl.
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Negative Numbers

Numbers with finite number of digits

- can be represented on a circle
- Addition
 - Clockwise
- Subtraction
 - Counter-clockwise



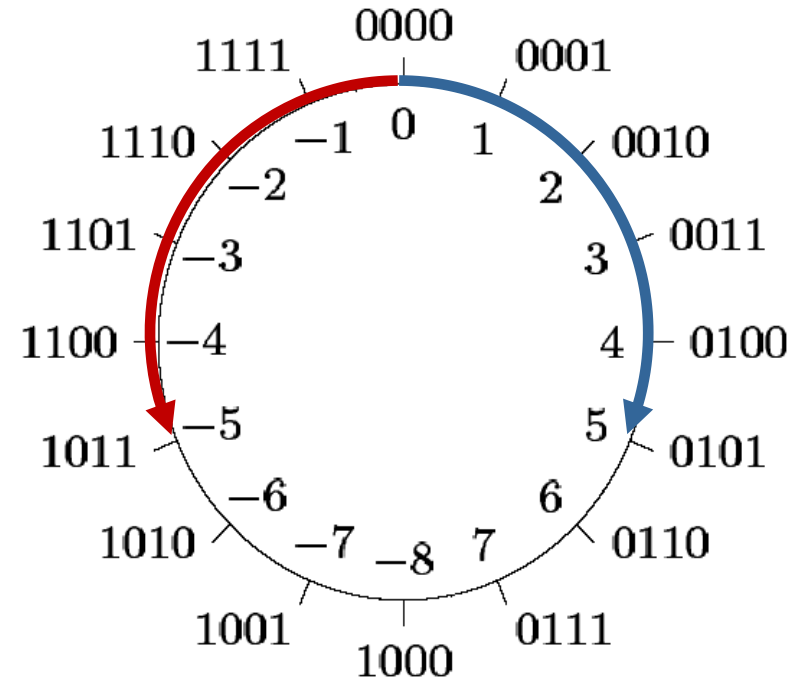
Negative Numbers

Negation of a number

- **Idea:** $-a = 0 - a$
- Starting at 0000 enter the absolute value of **a** counter-clockwise
- Example: $5d = 0101 \rightarrow -5d = 1011$

written calculation

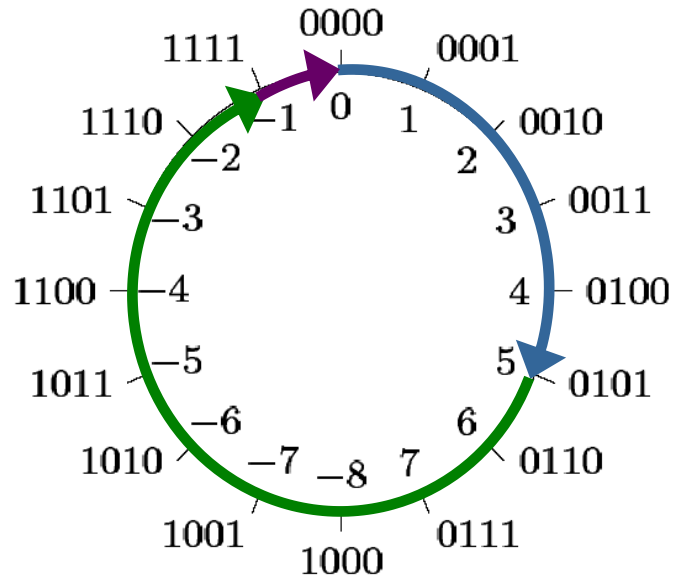
$$\begin{array}{r} 0000 \\ - 0101 \\ \hline 1111 \\ \hline 1011 \\ \hline \hline \end{array}$$



Negative Numbers

2' Complement

$$a - a = 0 \leftrightarrow a + \text{OC}(a) + 1 = 0$$

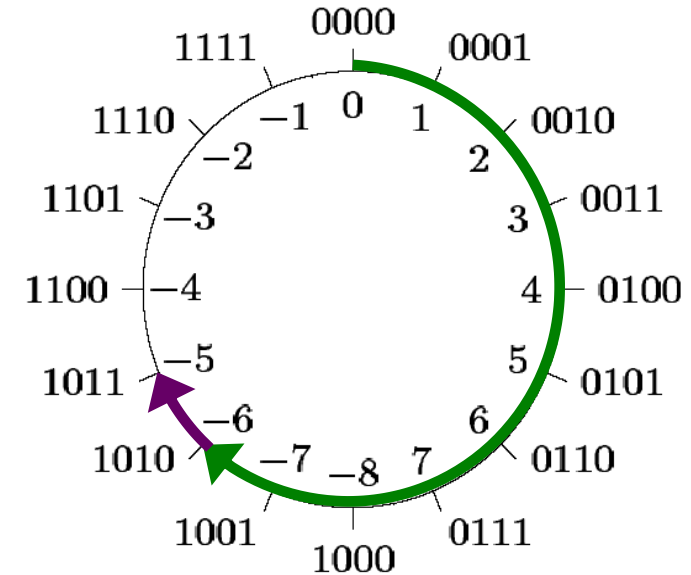


OC(a) is the bit-wise inverse of a

1111	
- 0101	
0 0 0 0	1010

OC(0101) = 1010

$$-a = \text{OC}(a) + 1 = \text{TC}(a)$$



$$-5d = \text{TC}(0101) = 1010 + 1 = 1011$$

OC: 1' complement
TC: 2' complement

Negative Numbers

Instruction rsbs

- Reverse Subtract
- Generates 2' complement
- Updates flags
- Only low register possible

```
rsbs rd, <rm>                                T1
rd, rm { r0..r7 }
```

010000	1001	nnn	ddd	N	Z	C	V
--------	------	-----	-----	---	---	---	---

rd = 0 - rn; or rd = 0 - rd;

00000022	4251	rsbs	r1,r2,#0	
00000024	427F	rsbs	r7,r7,#0	
00000026	427F	rsbs	r7,#0	@ the same (dest = r7)
00000028		@rsbs	r8,r1,#0	@not possible (high reg)
00000028		@rsbs	r1,r8,#0	@not possible (high reg)

Negative Numbers

Example: Instruction rsbs

C-Code

```
int32_t intA = 3;
int32_t intB;

int32_t invertSign(void)
{
    ...
    intB = -intA;
    ...
}
```

Assembler-code

```
.text
...
ldr    r0,=intA      ①
ldr    r0,[r0,#0]
rsbs    r0,r0,#0
ldr    r1,=intB      ①
str    r0,[r1,#0]
...

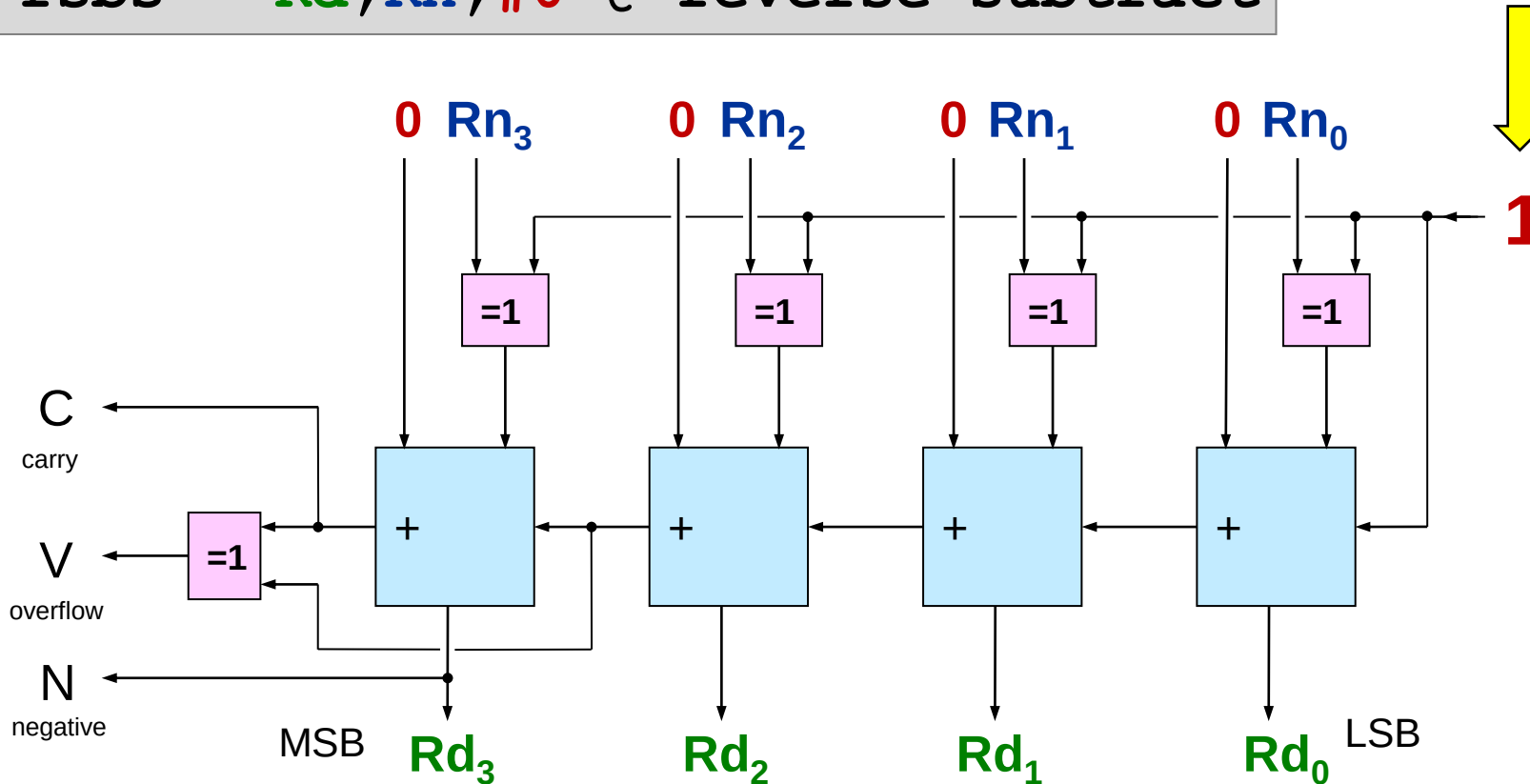
.data
intA    .word 0x00000003
intB    .word 0x00000000
```

① The compiler does not use the pseudo-instruction ldr in this case

Negative Numbers

2' complement in Hardware (ALU)

rsbs **Rd**, **Rn**, #0 @ reverse subtract



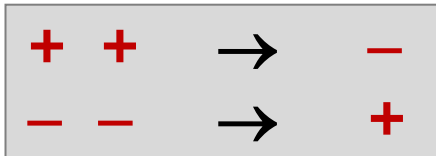
Addition

– Unsigned

- C = 1 indicates carry
- V irrelevant
- Addition of 2 big numbers
→ can yield a small result

– Signed

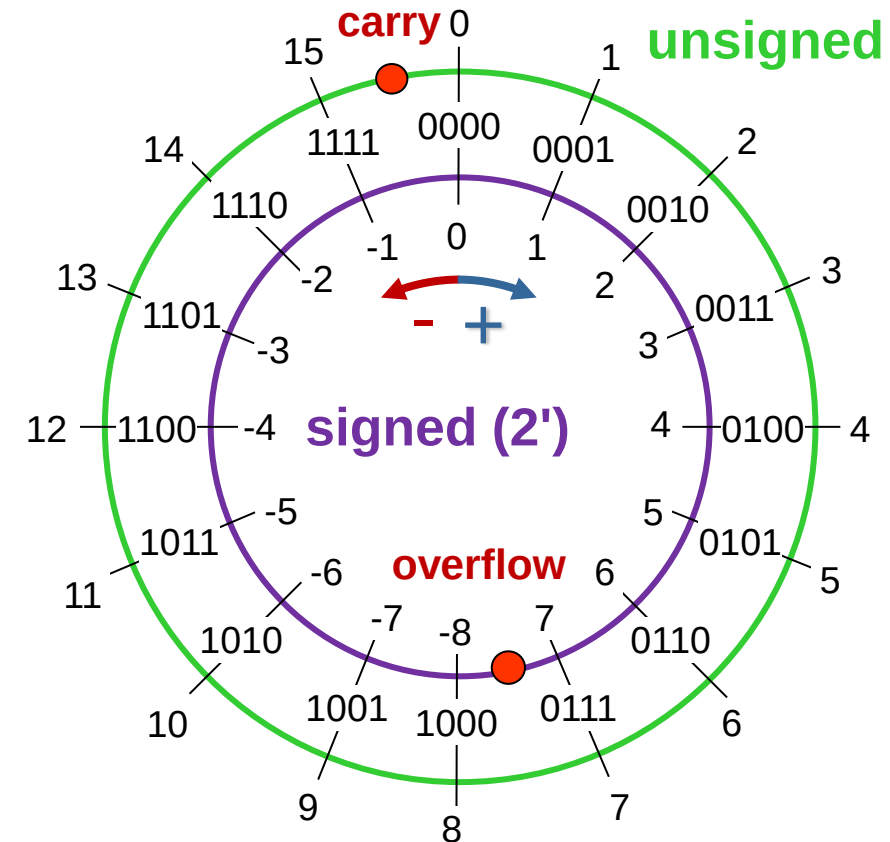
- V = 1 indicates overflow
 - possible with same „sign“



- C irrelevant

Programmer interprets register content
either as signed or as unsigned.
CPU does not care!

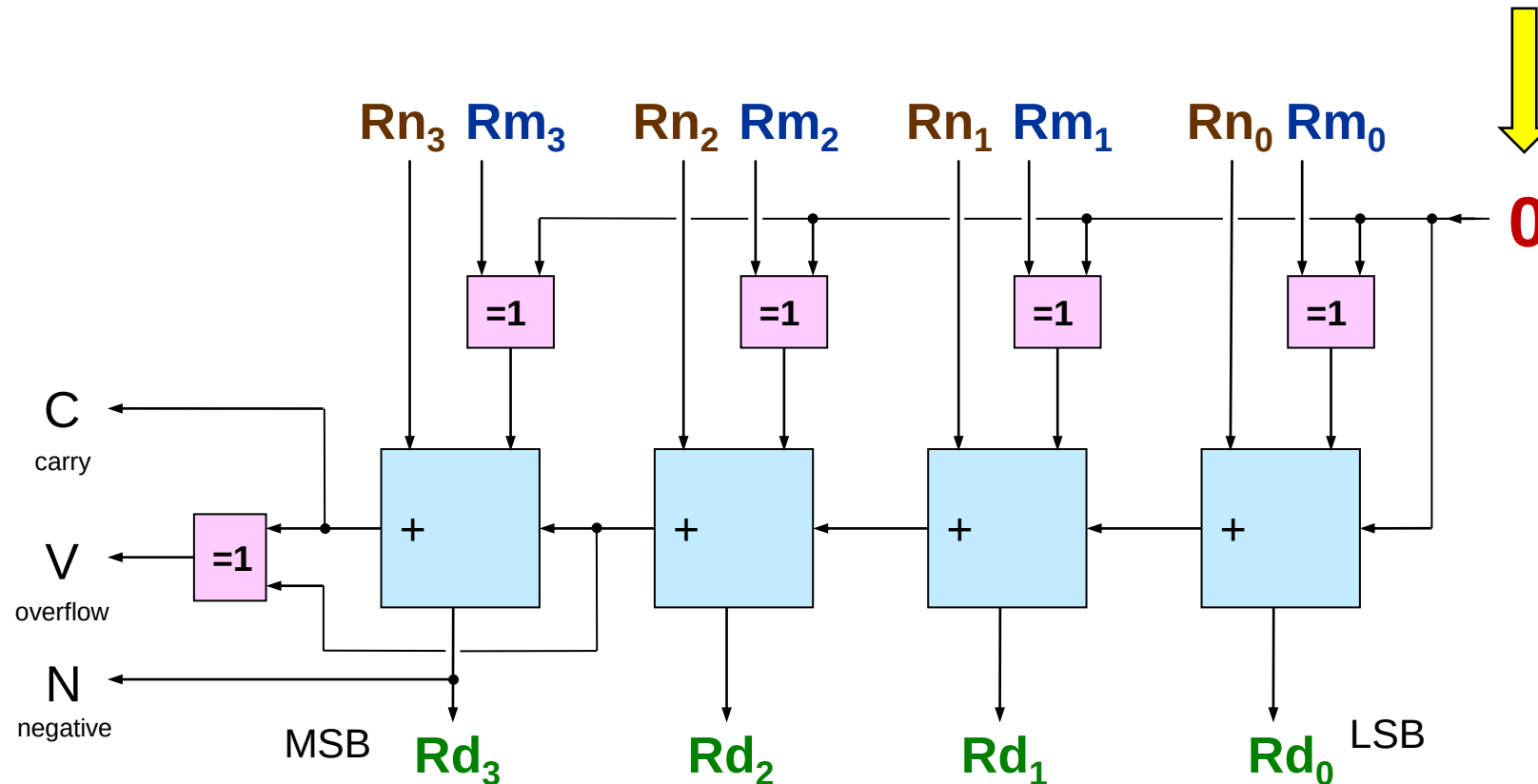
```
adds r1,r2,r3
```



Addition

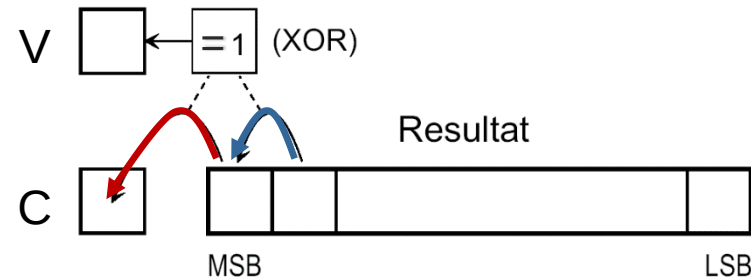
Addition in Hardware (ALU)

adds **Rd**, **Rn**, **Rm**



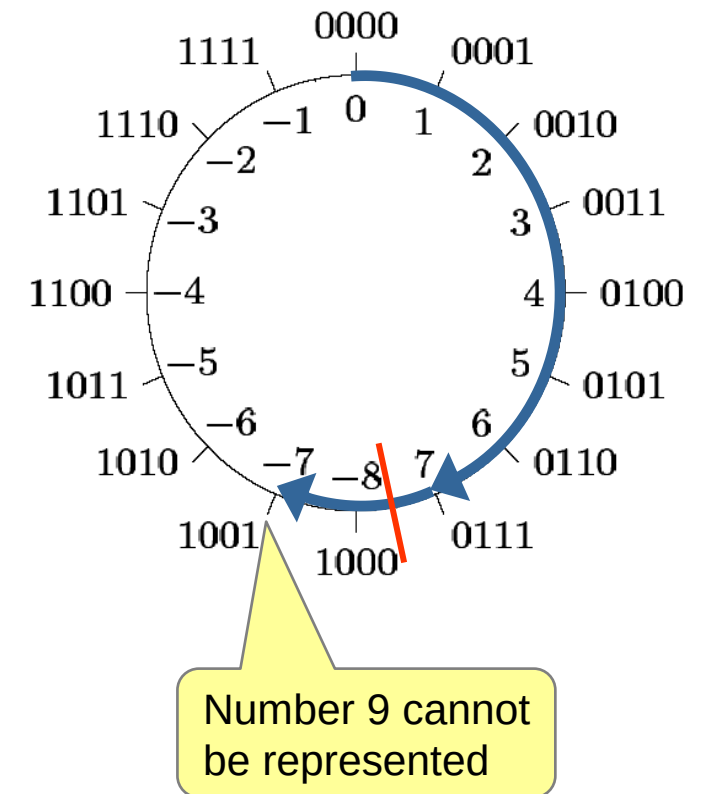
Addition Signed (1)

- Overflow carry to MSB has different value than carry from MSB



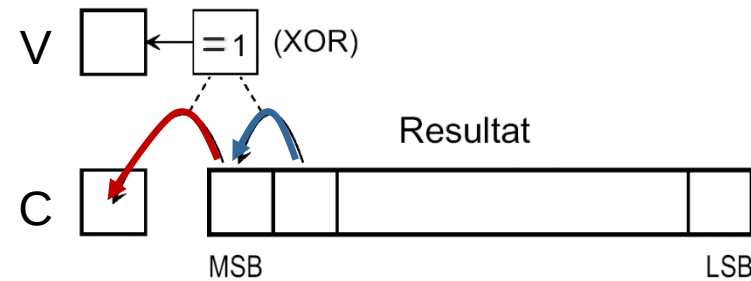
- Examples for 4 cases

0 1 0111 +0010 0 1 1 0 1001 V = 1	1 0 1010 +1101 1 0 0 0 0111 V = 1	0 0 0001 +1110 0 0 0 0 1111 V = 0	1 1 0011 +1111 1 1 1 1 0010 V = 0
--	--	--	--



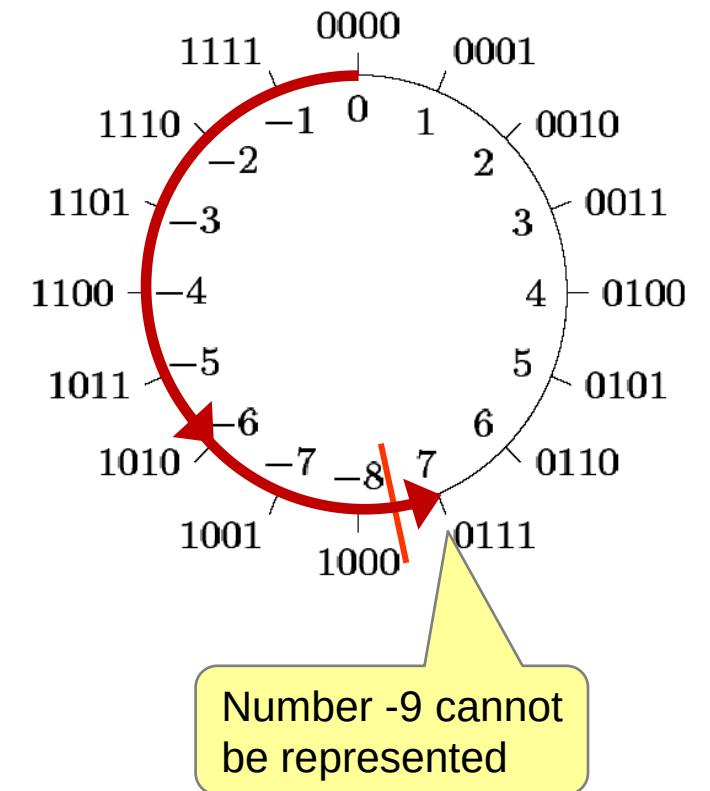
Addition Signed (2)

- Overflow carry to MSB has different value than
 carry from MSB



- Examples for 4 cases

0 1	1 0	0 0	1 1
0111	1010	0001	0011
+0010	+1101	+1110	+1111
<u>0 1 1 0</u>	<u>1 0 0 0</u>	<u>0 0 0 0</u>	<u>1 1 1 1</u>
<u>1001</u>	<u>0111</u>	<u>1111</u>	<u>0010</u>
V = 1	V = 1	V = 0	V = 0

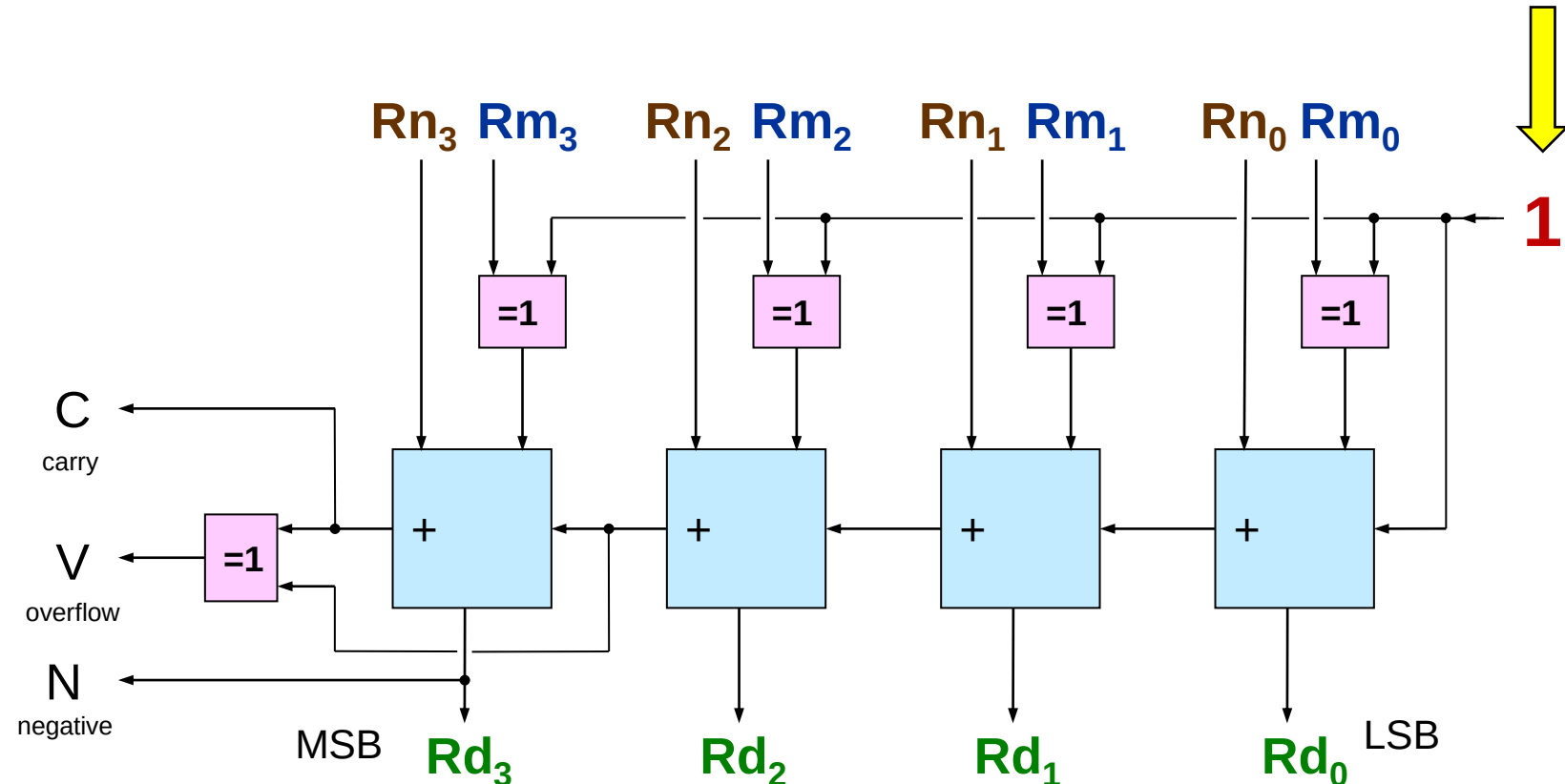


Subtraction

Subtraction in Hardware (ALU)

- There is no subtraction!
 - use addition of 2' complement instead

subs **Rd**, **Rn**, **Rm**

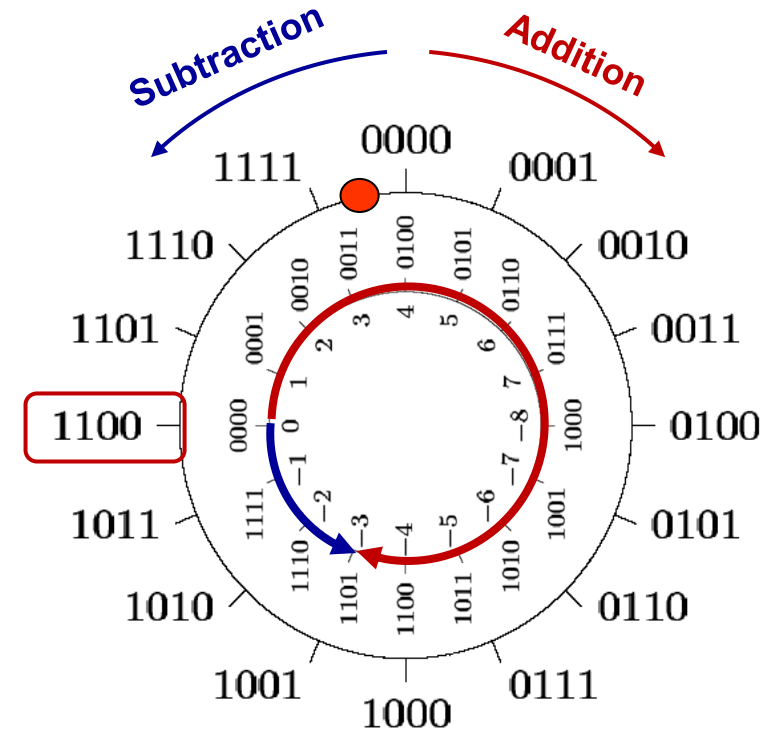


Subtraction unsigned

- Use 2' complement as well
- Example 4-Bit unsigned: $12 - 3 = 9$

12	1100	1100
- 3	- 0011	+ 1101
		1 1
9	1001	1 1001
	human	computer

- Attention
 - $C = 1 \rightarrow$ NO borrow
 - $C = 0 \rightarrow$ borrow
- Borrow
 - operation yields negative result $\rightarrow$ cannot be represented in unsigned
 - in multi-word operations missing digits are borrowed from more significant word



Subtraction Summary

– Unsigned

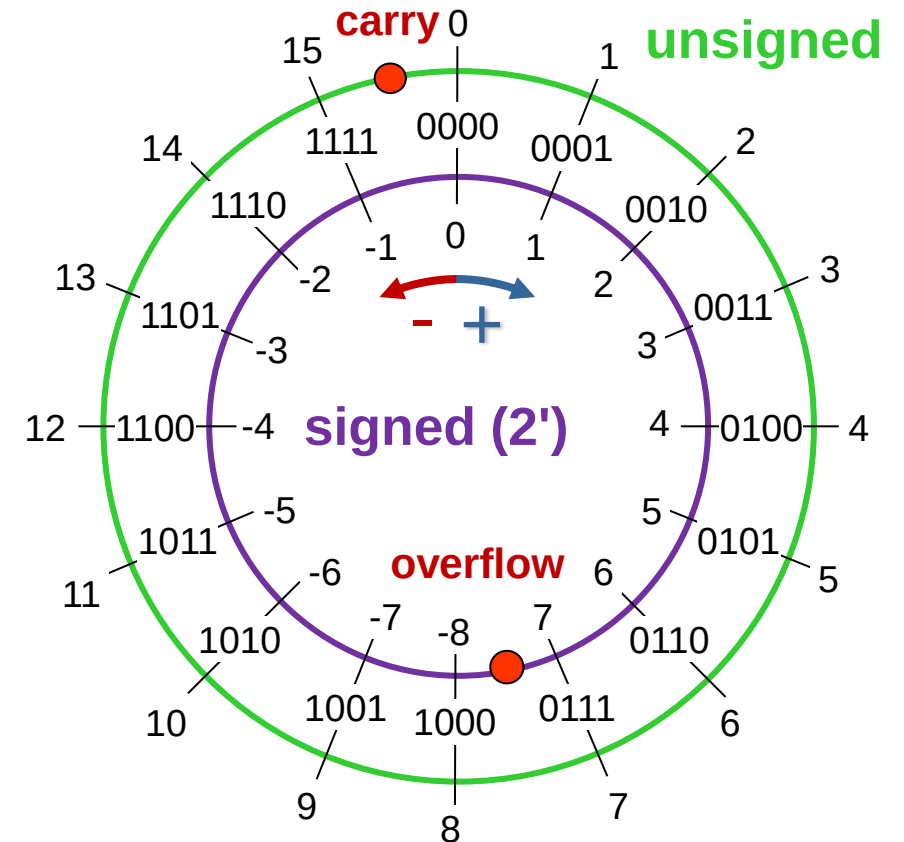
- C = 0 indicates borrow
- V irrelevant
- Subtraction from a small number
→ can yield a big result

– Signed

- V = 1 indicates overflow or underflow
 - Possible with opposite signs
 - NOT possible when operands have same sign
- C irrelevant

Programmer interprets register content
either as signed or as unsigned.
CPU does not care!

```
subs r1,r2,r3
```



Subtraction Operations Cortex-M0

subs (register)

- Updates flags
- Result and 2 operands
- Only low register

```
subs rd, <rn>, rm                                T1
rd, rn, rm { r0..r7 }
```

000	11	0	1	mmm	nnn	ddd	N	Z	C	V
-----	----	---	---	-----	-----	-----	---	---	---	---

```
rd = rn - rm; or rd -= rm;
```

00000016	1AD1	subs	r1,r2,r3	
00000018	1B64	subs	r4,r4,r5	
0000001A	1B64	subs	r4,r5	@ the same (dest = r4)
0000001C				
0000001C		@subs	r8,r4,r5	@ not possible (high reg)
0000001C		@subs	r4,r8,r5	@ not possible (high reg)
0000001C		@subs	r4,r5,r8	@ not possible (high reg)

Subtraction Operations Cortex-M0

subs (immediate) – T1

- Updates flags
- 2 different low registers and immediate value 0 - 7d

```
subs rd, rn, #imm3                                T1
rd, rn ∈ { r0..r7 }
imm3 ∈ { 0..7 }
```

000	11	1	1	iii	nnn	ddd		N	Z	C	V
-----	----	---	---	-----	-----	-----	--	---	---	---	---

```
rd = rn - imm3; or rd -= imm3;
```

0000001C 1F63	subs	r3,r4,#5	
0000001E	@subs	r3,r4,#8	@ out of range immediate
0000001E	@subs	r10,r11,#5	@ not possible (high reg)

Subtraction Operations Cortex-M0

subs (immediate) – T2

- Updates flags
- One low register and immediate value 0 - 255d
- <Rdn> → Result and operand
 - → same register for result and operand

```
subs rd, #imm8                                T2
rd ∈ { r0..r7 }
imm8 ∈ { 0..255 }
```

001	11	ddd	iiiiiii	
-----	----	-----	---------	--

N Z C V

rd = rd - imm8;

0000001E	3BF0	subs	r3,r3,#240	
00000020	3BF0	subs	r3,#240	@ the same (dest = r3)
00000022		@subs	r8,r8,#240	@ not possible (high reg)
00000022		@subs	r3,#260	@ out of range immediate

Addition / Subtraction

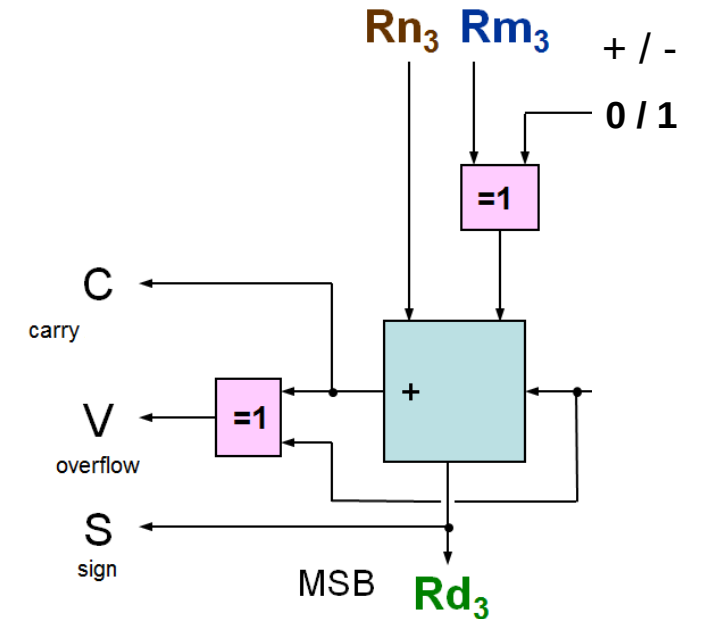
Interpretation of carry flag

– Addition $C = 1 \rightarrow$ Carry

1	1	0	1	13d
0	1	1	1	7d
1	1	1	1	
1	0	1	0	0
				20d $\rightarrow$ 16d + 4d

– Subtraction $C = 0 \rightarrow$ Borrow

6d - 14d = 0110b - 1110b = 0110b + 0010b				
0	1	1	0	6d
0	0	1	0	2d = TC(14d)
0	1	1	0	
0	1	0	0	0
				8d $\rightarrow$ - 16d + 8d



Addition / Subtraction

Interpretation of flags

- **unsigned** Interpretation
 - Program must check **carry flag (C)** after operation
 - **C = 1 for Addition** **C = 0 for Subtraction**
 - Result cannot be represented (not enough digits / no negative numbers)
 - Full turn on number circle must be added or subtracted
→ odometer effect
 - Overflow flag (V) irrelevant
- **signed** Interpretation
 - Program must check **overflow flag (V)** after operation
 - **V = 1** means
 - Not enough digits available to represent the result
 - Full turn on number circle must be added or subtracted
→ odometer effect
 - Carry flag (C) irrelevant

Multi-Word Arithmetic

Addition adcs

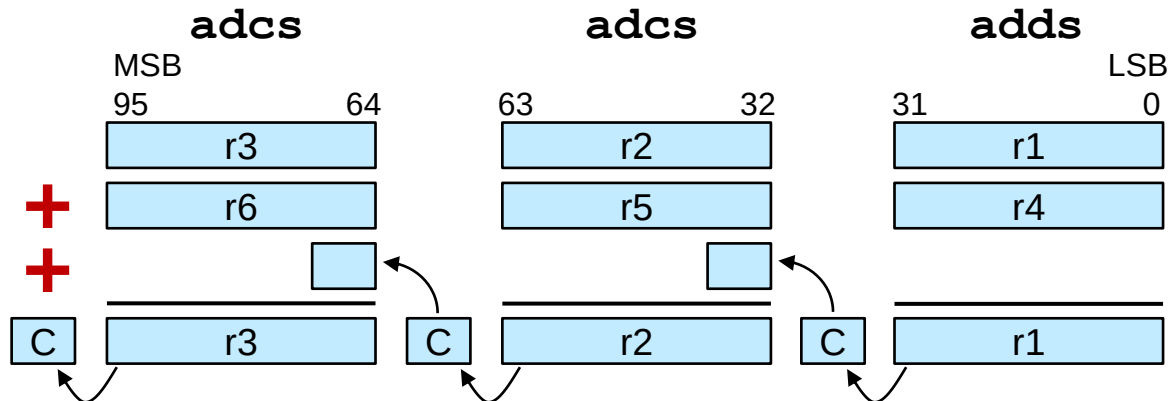
- Example: Addition of two 96-bit Operands

adcs rd, rm *T1*
 rd, rm ∈ { r0..r7 }

010000	0101	mmm	ddd
--------	------	-----	-----

N Z C V

rd = rd + rm + carry;



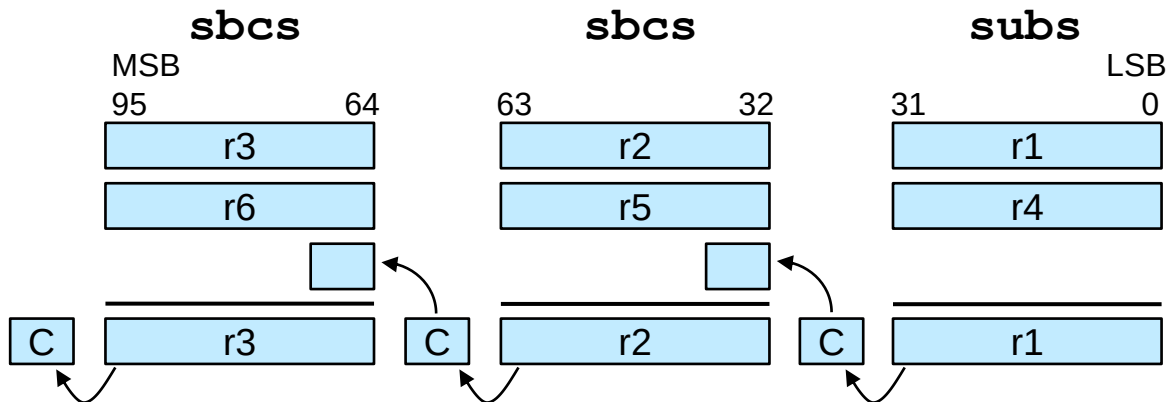
```
@ Operand A:      96-bit in r3 (MSW) ,r2,r1 (LSW)
@ Operand B:      96-bit in r6 (MSW) ,r5,r4 (LSW)
@ Result = A + B:  96-bit in r3 (MSW) ,r2,r1 (LSW)

00000028 1909      adds    r1,r1,r4
0000002A 416A      adcs    r2,r2,r5
0000002C 4173      adcs    r3,r3,r6
```

Multi-Word Arithmetic

Subtraction sbcs

- Example: Subtraction of two 96-bit Operands



```

@ Operand A:      96-bit in r3 (MSW) ,r2 ,r1 (LSW)
@ Operand B:      96-bit in r6 (MSW) ,r5 ,r4 (LSW)
@ Result A - B:   96-bit in r3 (MSW) ,r2 ,r1 (LSW)
0000002E 1B09      subs      r1, r1, r4
00000030 41AA      sbcs      r2, r2, r5
00000032 41B3      sbcs      r3, r3, r6
    
```

```

sbcs rd, rm T1
rd, rm { r0..r7 }

010000 0110 mmm ddd N Z C V

rd = rd - rm - (-carry);
    
```

¹⁾ $-Rm - NOT(C)$
 $= (NOT(Rm) + 1) - NOT(C)$
 $= NOT(Rm) + C$

The CPU actually calculates
 $Rdn + NOT(Rm) + C$

Multiplication

mul (register)

- Flags
 - N and Z updated
 - C and V unchanged
- Operands
 - Rn multiplicand
 - Rdm multiplier
 - only low registers
- Result
 - Rdm contains only lowest 32 bits of product

```
mul rd, rn T1  
rd, rn ∈ { r0..r7 }
```

010000	1101	nnn	ddd	N Z
--------	------	-----	-----	-----

```
rd = rd * rm;
```

```
0000002E 4351 mul r1,r2,r1
```

```
00000030      @mul r1,r1,r2 @ not possible: destination  
00000030                        @ and 2nd source must be same  
00000030      @mul r1,r8,r1 @ not possible: high reg
```

Multiplication

Size of Result

- Result requires twice as many binary digits
 - Example 4-bit * 4-bit → 8-bit result
- Signed and unsigned multiplication are different

```

0101 * 0011
-----
      0011
     0000
    0011
   0000
  -----
00001111

```

Interpretation **unsigned**
 $5d * 3d = 15d \rightarrow \text{correct}$

Interpretation **signed**
 $5d * 3d = 15d \rightarrow \text{correct}$

unsigned

```

0101 * 1101
-----
 00001101
00000000
 00001101
00000000
  -----
0000100001

```

zero extension
of multiplier

Interpretation **unsigned**
 $5d * 13d = 65d \rightarrow \text{correct}$

Interpretation **signed**
 $5d * (-3d) = 65d \rightarrow \text{wrong}$

signed

```

0101 * 1101
-----
 11111101
00000000
 11111101
00000000
  -----
1001111001

```

sign extension
of multiplier

Interpretation **unsigned**
 $5d * 13d = 241d \rightarrow \text{wrong}$

Interpretation **signed**
 $5d * (-3d) = -15d \rightarrow \text{correct}$

Multiplication mul on Cortex-M0

- R_n (32-bit) * R_{dm} (32-bit) $\rightarrow$ R_{dm} (32-bit)
 - Upper 32-bit of result are lost!
 - Lower 32-bit are the same for unsigned and signed
- unsigned $\rightarrow$ watch out if result requires more than 32 bits
- signed $\rightarrow$ part with sign bit is missing

Conclusion

- **Processor Arithmetic**
 - Odometer effect because of finite word length
- **Processors do **not** distinguish signed and unsigned**
 - Users (resp. compilers) have to know, whether they are working with signed or unsigned numbers
 - Processor always calculates flags for both cases
 - carry (C) unsigned
 - overflow (V) signed
negative (N)

Conclusion

- **unsigned**

- Addition → $C = 1 \rightarrow$ carry
result too large for available bits
- Subtraction → $C = 0 \rightarrow$ borrow
result less than zero → no negative numbers

- **signed**

- Addition → potential overflow in case of operands with equal signs
- Subtraction → potential overflow in case of operands with opposite signs

- **Arithmetic Instructions**

- **add/adds adcs adr**
sub/subs sbcs rsbs
muls